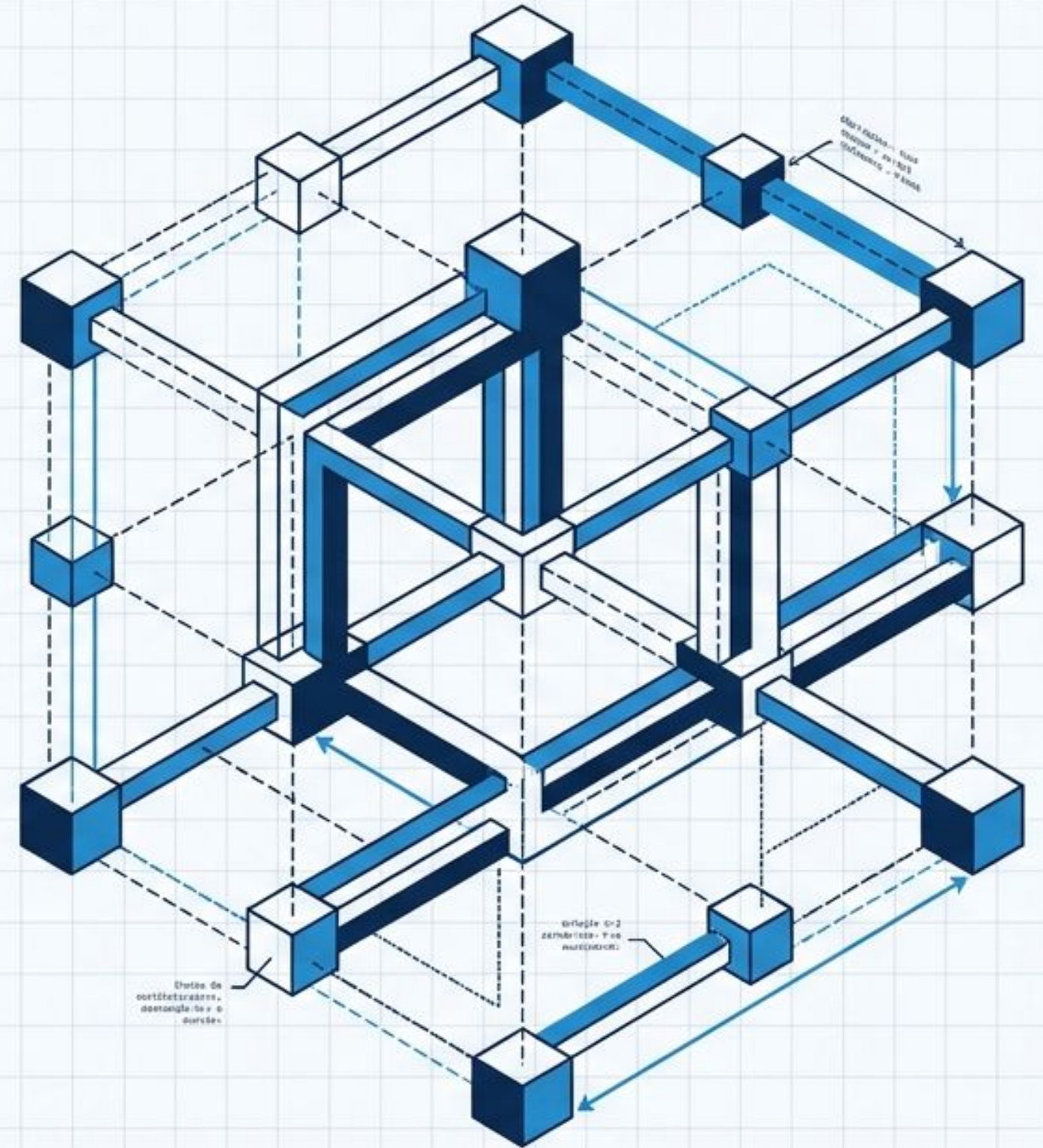


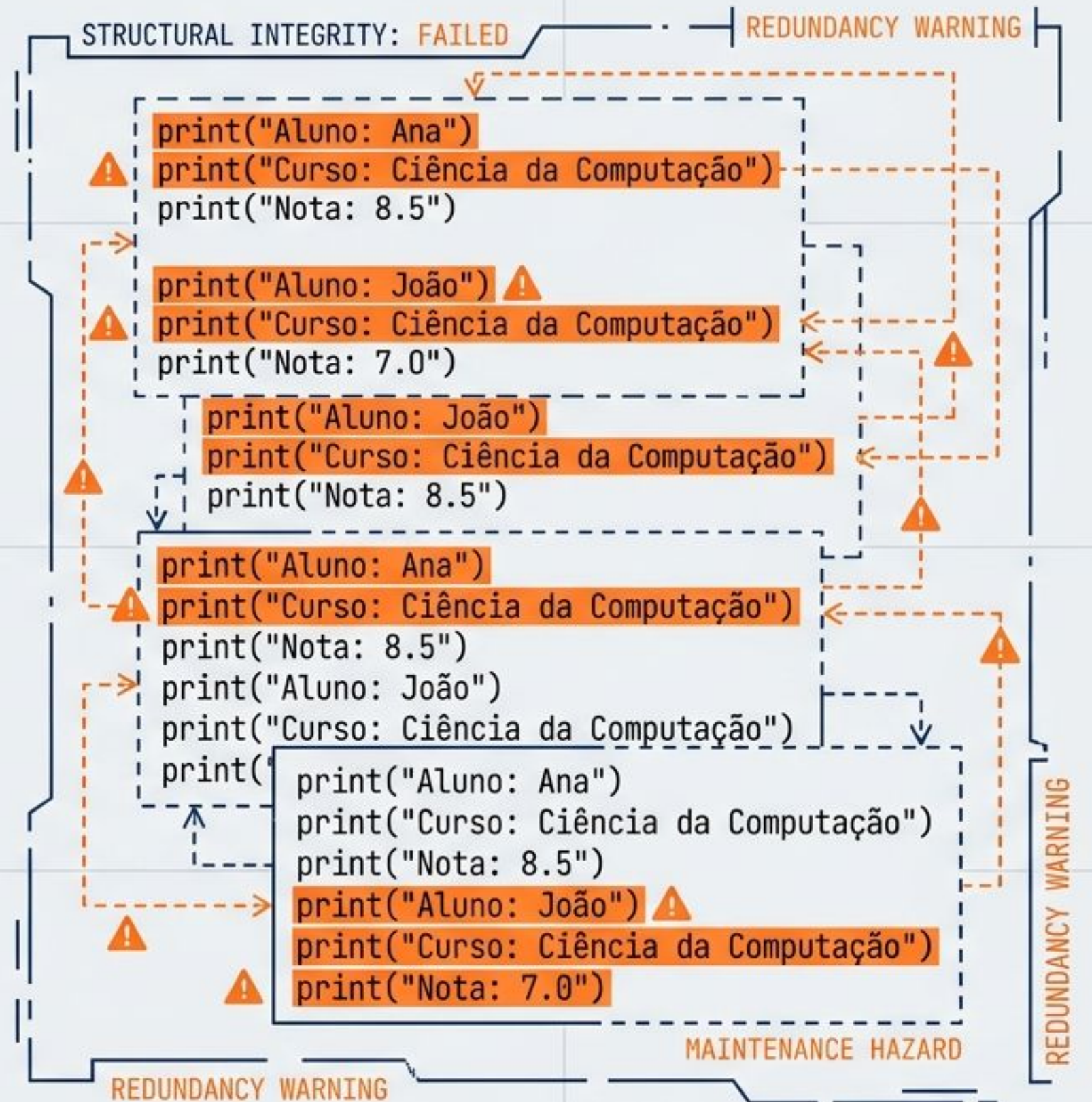
Arquitetura de Código com Python

Dominando Funções:
Modularização, Abstração
e a Arte de Dividir para
Conquistar.



O Caos da Repetição

Sem funções, o código cresce de forma desordenada. A repetição contínua gera extrema dificuldade de leitura, testes complexos e uma manutenção quase impossível.



O Conceito: Blocos de Construção

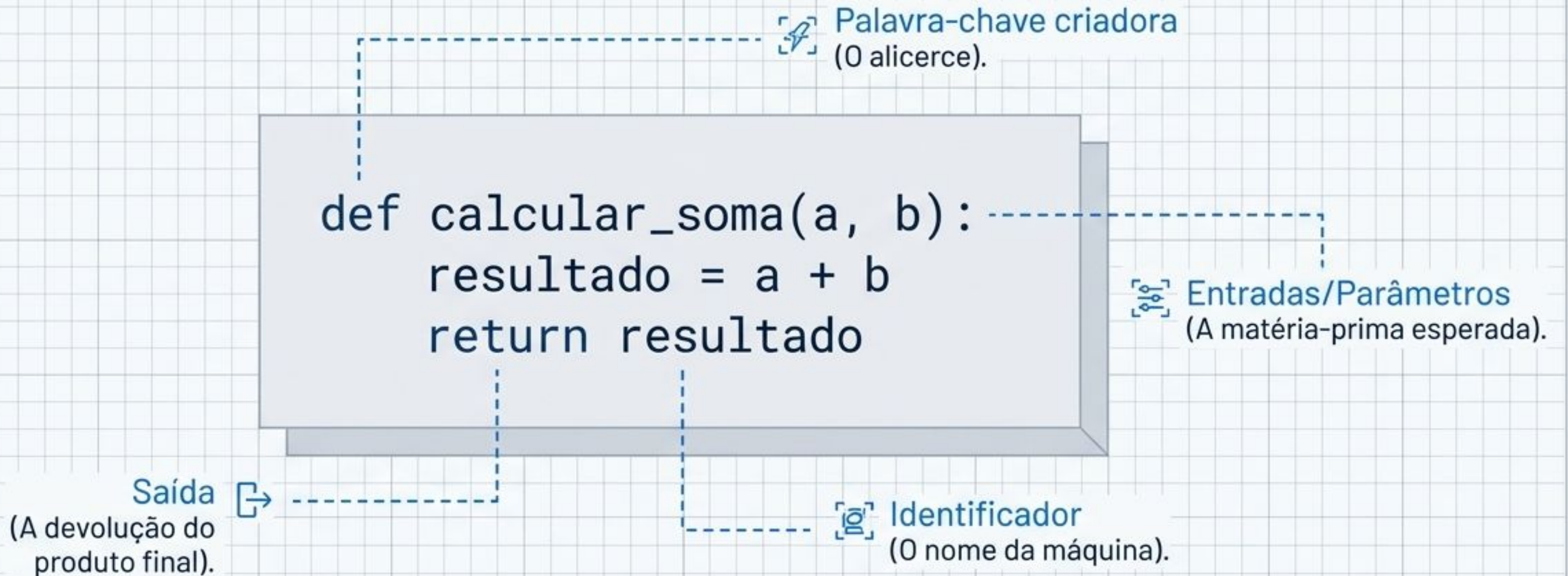
Uma função é uma "máquina" criada para executar uma tarefa específica e transformá-la em um componente reutilizável do seu programa.



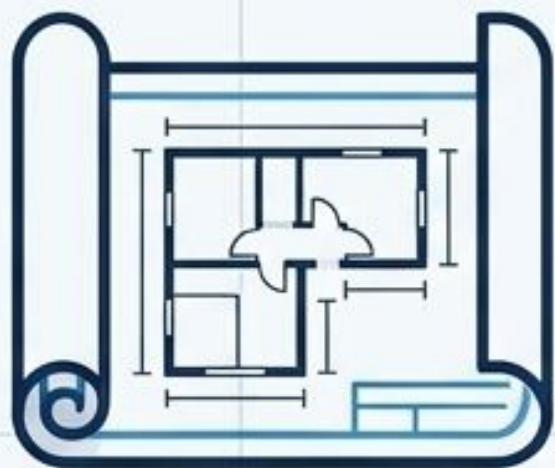
Por que projetar com funções?



A Anatomia do Código



Definição vs. Invocação



→ `def saudacao():`

Criar a Função.

Ensina o Python a fazer algo. Nenhum código é executado neste momento; é apenas um projeto aguardando uso.

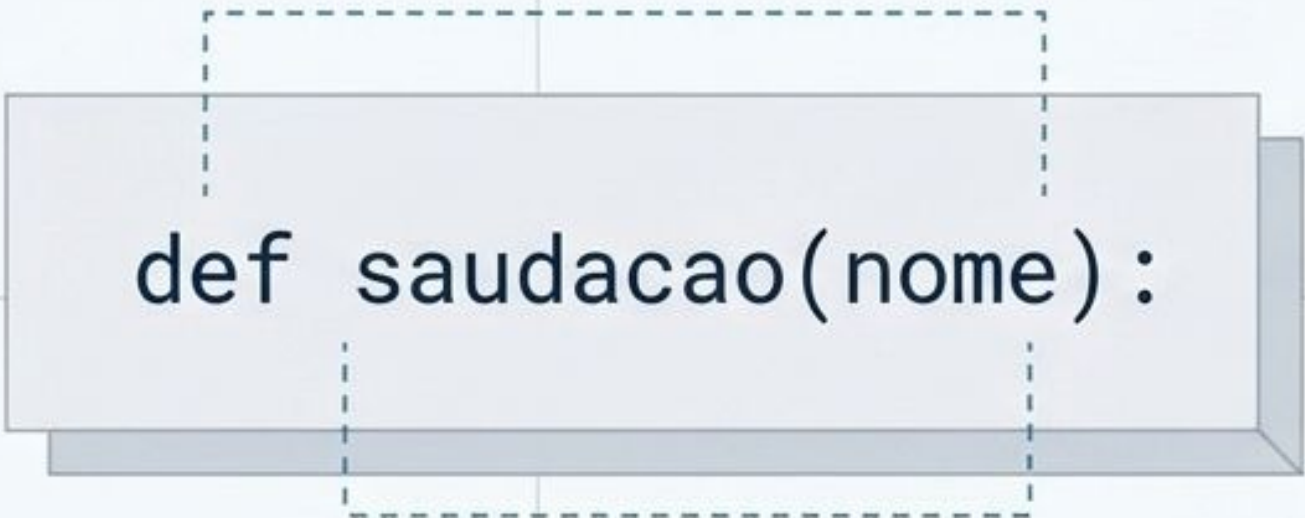
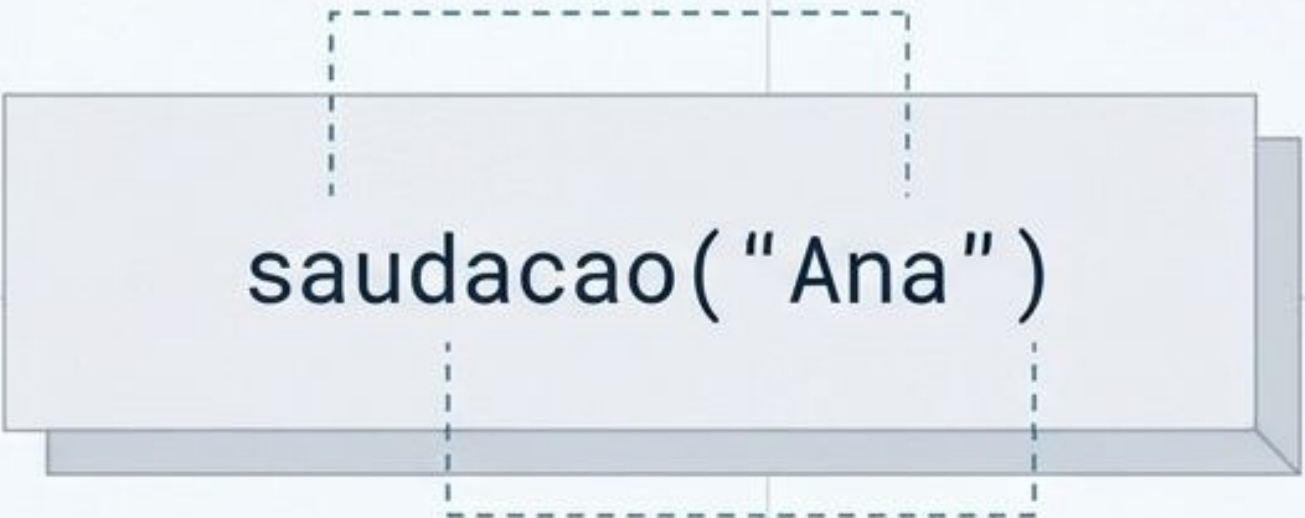


`saudacao()` ←

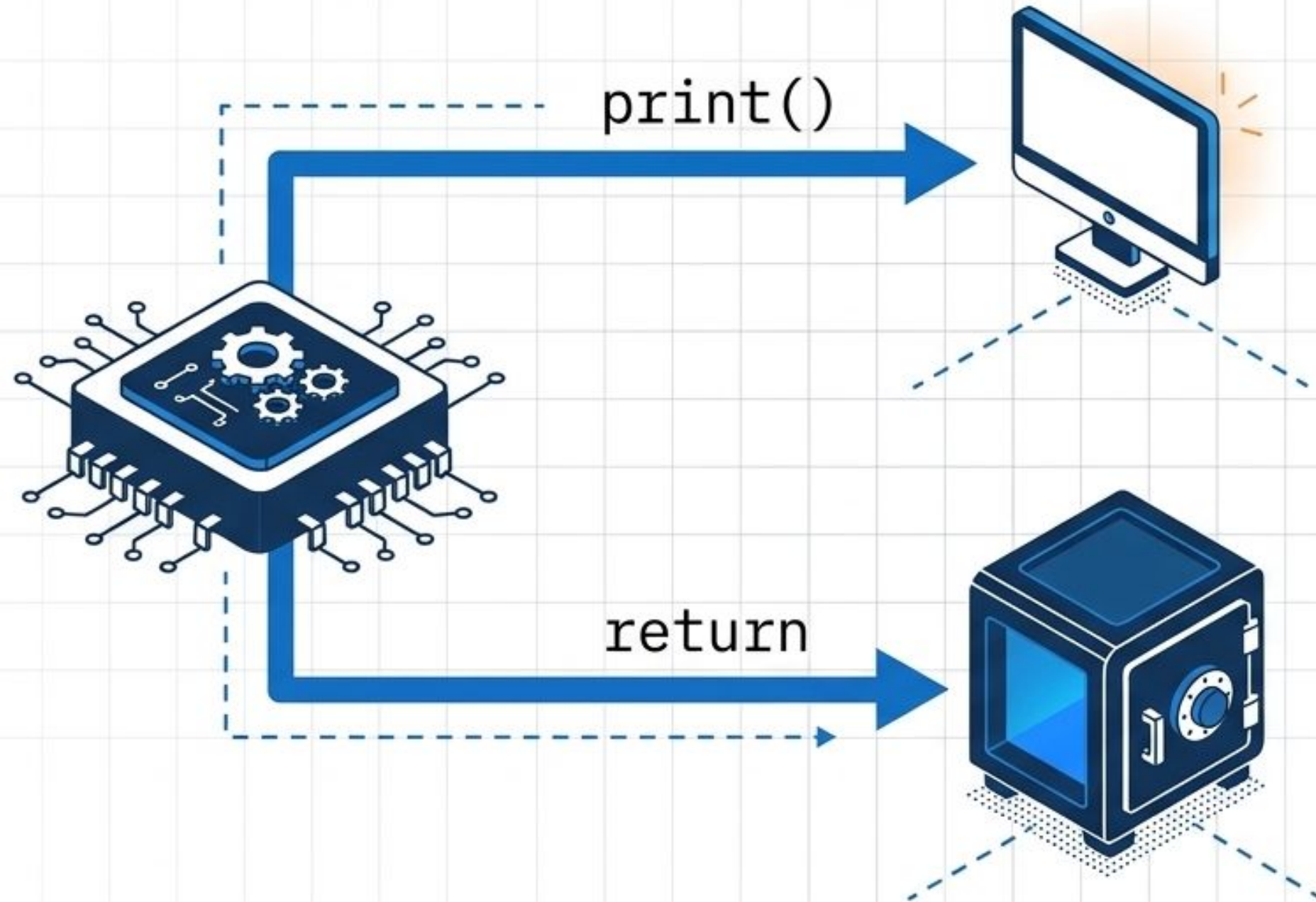
Chamar a Função.

Aciona a máquina para trabalhar de fato, disparando o bloco de código interno.

Desfazendo a Confusão

Parâmetro (O Molde)	Argumento (A Matéria-Prima)
É a variável definida no momento da criação da função. Fica aguardando para receber um valor.  <pre>def saudacao(nome):</pre> The diagram shows a light blue box containing the code <code>def saudacao(nome):</code>. A dashed rectangle highlights the parameter <code>nome</code>. A dashed line extends from the top of this box to the text above, and another dashed line extends from the bottom of the box to the text below.	É o valor real e concreto enviado no momento da chamada da função.  <pre>saudacao("Ana")</pre> The diagram shows a light blue box containing the code <code>saudacao("Ana")</code>. A dashed rectangle highlights the argument <code>"Ana"</code>. A dashed line extends from the top of this box to the text above, and another dashed line extends from the bottom of the box to the text below.

O Grande Duelo: Exibir vs. Devolver



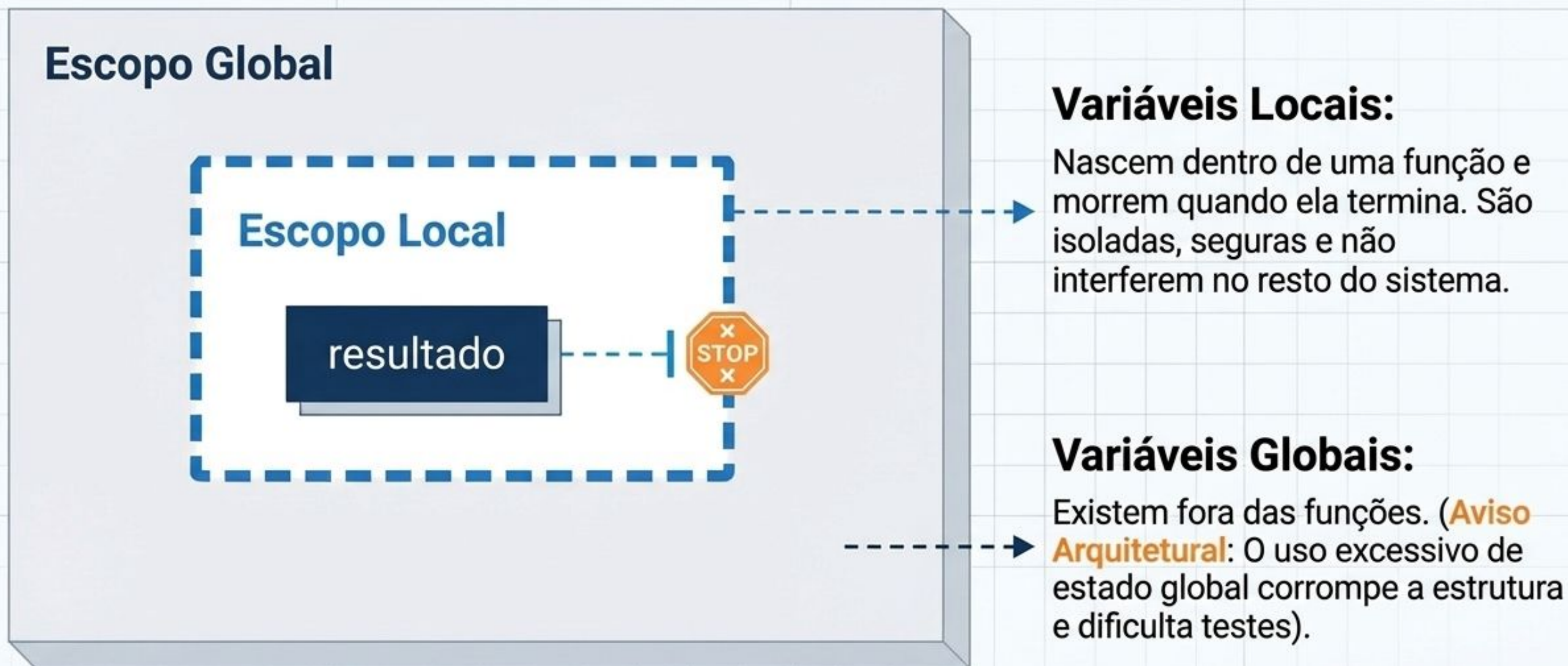
Mostra na tela.

É apenas um reflexo visual para o usuário. O programa perde o valor imediatamente após exibi-lo (retorna None).

Devolve o valor.

O programa captura o resultado internamente, permitindo armazená-lo em variáveis ou utilizá-lo em novos cálculos.

O Território das Variáveis (Escopo)



Polimento Profissional

```
def somar(a, b):
```



```
def somar(a: int, b: int) -> int:
```

Type Hints

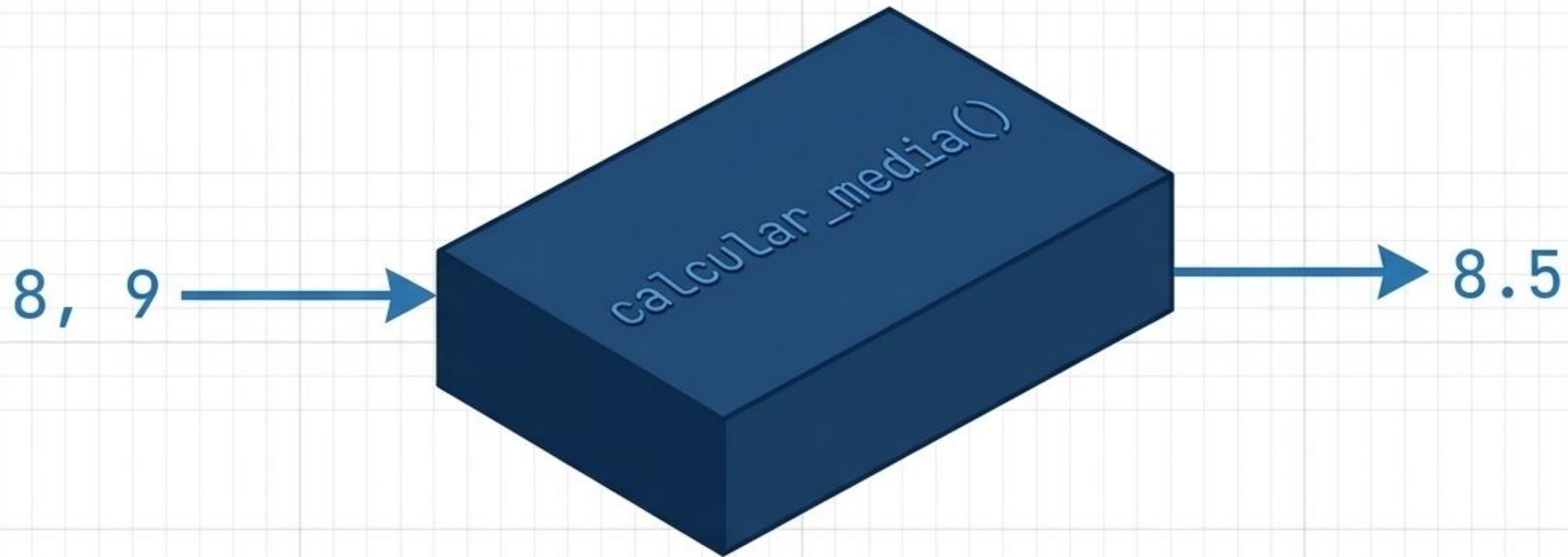
Não são barreiras rígidas em Python, mas funcionam como a documentação oficial da sua arquitetura, garantindo previsibilidade.

Valores Padrão

```
def saudacao(nome="estudante"):
```

Oferece flexibilidade máxima. Se o argumento não for fornecido na chamada, a máquina assume um comportamento padrão seguro.

A Arte da "Caixa Preta" (Abstração)



Você não precisa saber como a função trabalha internamente. Não é necessário pensar em "somar" ou "dividir por 2". Para o engenheiro de software, basta conhecer a entrada que a máquina exige e a saída garantida que ela entrega. Isso reduz drasticamente a carga mental ao projetar sistemas complexos.

Composição: A Linha de Montagem

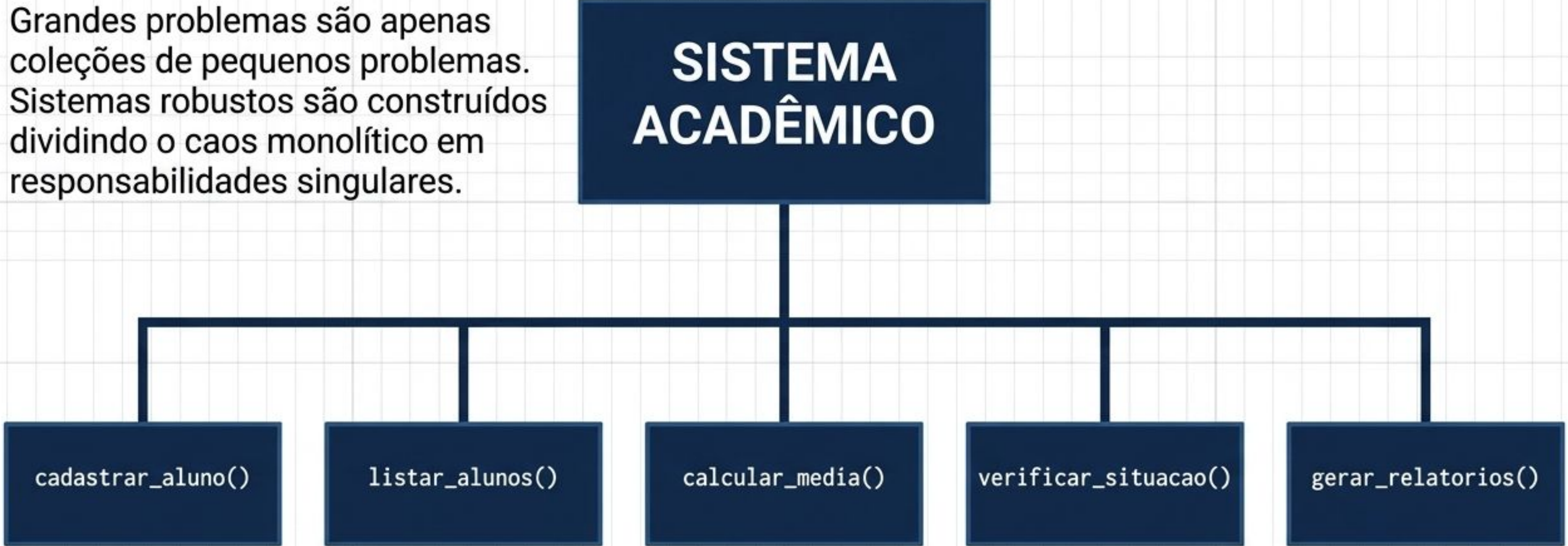


O verdadeiro poder surge quando combinamos as peças. O output (saída) de uma máquina torna-se imediatamente o input (entrada) da próxima, criando fluxos de dados automatizados e contínuos.

Dividir para Conquistar (Decomposição)

Grandes problemas são apenas coleções de pequenos problemas. Sistemas robustos são construídos dividindo o caos monolítico em responsabilidades singulares.

SISTEMA ACADÊMICO



```
graph TD; SA[SISTEMA ACADÊMICO] --- C[cadastrar_aluno()]; SA --- L[listar_alunos()]; SA --- M[calcular_media()]; SA --- V[verificar_situacao()]; SA --- G[gerar_relatorios()];
```

O diagrama ilustra a decomposição do 'SISTEMA ACADÊMICO' em cinco funcionalidades distintas. No topo, um retângulo escuro azul contém o título 'SISTEMA ACADÊMICO' em letras brancas. Uma linha vertical desce deste retângulo e se conecta a uma barra horizontal. A partir desta barra, cinco linhas verticais descendentes ligam o sistema principal a cinco retângulos escuros azuis dispostos horizontalmente na base. Cada um destes retângulos contém o nome de uma função em código: 'cadastrar_aluno()', 'listar_alunos()', 'calcular_media()', 'verificar_situacao()' e 'gerar_relatorios()'.

`cadastrar_aluno()`

`listar_alunos()`

`calcular_media()`

`verificar_situacao()`

`gerar_relatorios()`

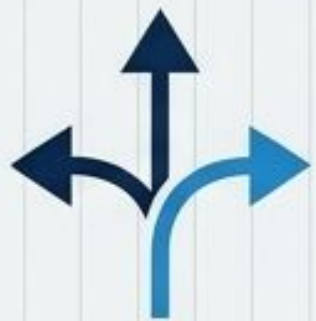
As Leis do Arquiteto



Responsabilidade Única. Uma função deve fazer apenas UMA coisa. Se você criou `cadastrar_e_calcular()`, divida-a.



Nomes Significativos. O nome deve ser a documentação. Fuja de `def f(x, y):`; exija `def calcular_media(nota1, nota2):`.



Tamanho Limitado. Evite funções gigantes. Se a função não cabe na tela do monitor, ela esconde múltiplos problemas não decompostos.

De Programador a Engenheiro

Dominar funções é o marco divisório. É o momento em que você deixa de simplesmente escrever "linhas de código" e passa a desenhar arquiteturas de software robustas e escaláveis.

Próximo Passo: O Desafio Final.

Projete seu próprio Sistema Acadêmico isolando as responsabilidades de cadastro, cálculo e relatórios em componentes 100% autônomos.

A teoria agora exige prática.

